Models And Databases in NodeJS

Purpose of a Model

A **model** defines how data is structured, validated, and stored in MongoDB. It acts as a **blueprint** for documents in a specific **collection. For** us to interact freely with a mongo database, we need a **mongoose library**.

What is Mongoose?

Mongoose is an **ODM (Object Data Modeling)** library for MongoDB and Node.js.It helps us **define the structure of documents**, **enforce validation**, and **interact easily** with the database.

How to install it

npm install mongoose

Here is how you import it:

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;
```

Creating a Schema

A **schema** defines the structure of documents in a collection. When defining a schema field, you can specify:

→ **Data type** (`type`)
→ **Validation rules** (`required`, `enum`, `match`, `min`, `max`)
→ **Formatting options** (`trim`, `lowercase`, `uppercase`)
→ **Defaults** (`default`)
→ **Relationships** (`ref`)

Below is an example of a Blog Schema, each field in the schema below has:

- ✓ **type** → specifies the data type (e.g., `String`, `Number`, `Date`).
- ✓ **required** → ensures the field must be filled before saving.
- ✓ `{ timestamps: true }` → automatically adds `createdAt` and `updatedAt` fields to each document.

```
const blogSchema = new Schema(
  {
    title: {
      type: String,
      required: true,
    },
    snippet: {
      type: String,
      required: true,
    },
    body: {
      type: String,
      required: true,
    },
  },
  { timestamps: true }
);
```

Once you are done defining your database and collection requirements you eventually create a model

Creating a Model

```
const Blog = mongoose.model('Blog', blogSchema);
module.exports = Blog;       You, 3 days ago • first
```

→ `mongoose.model()` turns the schema into a **model**.
→ The first argument `"Blog"`: names the model.Mongoose automatically creates a **collection** called `blogs` (lowercase + pluralized).
→ The second argument is the schema that defines the structure.
→ You export the model to make it reusable in other files, e.g., controllers or routes.

What is a .env File?

A .env file (short for environment file) is a simple text file used to store environment variables key-value pairs that configure your app without hardcoding sensitive or changeable information inside your code.

A `.env` file keeps your app's sensitive information safe and makes your project easier to configure across different environments — all without ever touching your main code.

Example of a .env file:

```
PORT=5000
MONGO_URI=mongodb+srv://studentadmin:StrongPass123@cluster0.mongodb.net/BlogPost
JWT_SECRET=mySuperSecretKey
DEBUG_MODE=true
```

Why We Use a .env File?

1. Security (Protects Sensitive Information)

You should never put credentials like database passwords, API keys, or secret tokens directly in your code. Instead, store them in a .env file so they are not exposed on GitHub or shared with others.

Instead of:

mongoose.connect("mongodb+srv://user:pass@cluster0.mongodb.net/db");

You write:

mongoose.connect(process.env.MONGO_URI);

And in your `.env` file:

MONGO_URI=mongodb+srv://user:pass@cluster0.mongodb.net/db

2. It makes Collaboration-Friendly

Team members can:

- Share code without sharing sensitive data.
- Each have their **own `.env` file** suited to their system.

The `.env` file should be listed in `.gitignore` so it's not pushed to GitHub.

3. Keeps Code Clean and Reusable

Instead of spreading configuration details throughout your code, all environment variables live in one central file.

Easier to maintain and modify later.

How to Use a .env File in Node.js

Step 1: Install dotenv

```
npm install dotenv
```

Step 2: Import and configure it in your main file (usually `server.js` or `app.js`)

```
import dotenv from "dotenv";
dotenv.config();
```

Step 3: Access your variables using `process.env`

```
const port = process.env.PORT || 3000;
const db = process.env.MONGO_URI;
```

`process.env` is a built-in Node.js object that stores environment variables.

Best Practices for Using `.env` Files

Rule	Description
Never commit it to GitHub	Add `.env` to your `.gitignore` file.
Use clear variable names	e.g., `DB_USER`, `DB_PASSWORD`, `API_KEY`.
Don't put spaces around =	Correct: `PORT=3000`, ❌ Wrong: `PORT = 3000`.
Use `.env.example` file	Share only variable names (not values) to show teammates what's needed.
Load it before using variables	Call `dotenv.config()` at the top of your app.

How We Work with SQL Databases in Node.js

When using **Node.js** with a **SQL database** (like MySQL, PostgreSQL, or SQLite), the main goal is to:

👉 **connect your Node app to the database** and
👉 **run SQL queries** (SELECT, INSERT, UPDATE, DELETE) from your code.

1. Install a Database Driver or ORM

There are two main ways to connect Node.js with an SQL database:

Method	Description	Example Library
Driver	Directly communicate with the database using SQL queries.	`mysql2`, `pg`, `sqlite3`
ORM (Object Relational Mapper)	Allows you to work with JavaScript objects instead of raw SQL.	`Sequelize`, `TypeORM`, `Prisma`

2. Using MySQL with Node.js (Raw SQL Example)

Step 1: Install MySQL driver

```
npm install mysql2
```

Step 2: Connect to your database

```
import mysql from "mysql2";

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "password123",
  database: "school_db"
});

db.connect(err => {
  if (err) throw err;
  console.log("Connected to MySQL database!");
});
```

Step 3: Run SQL Queries

```
// Create table
db.query(
  "CREATE TABLE IF NOT EXISTS students (id INT AUTO_INCREMENT PRIMARY
KEY, name VARCHAR(255), age INT)",
  (err, result) => {
    if (err) throw err;
    console.log("Table created!");
  }
);

// Insert data
db.query("INSERT INTO students (name, age) VALUES (?, ?)", ["Alice",
22], (err, result) => {
  if (err) throw err;
  console.log("Record inserted!");
});

// Select data
db.query("SELECT * FROM students", (err, results) => {
```

```
  if (err) throw err;
  console.log(results);
});
```

Explanation:

- `db.query()` runs SQL commands directly.
- You can use `?` placeholders to safely insert values (avoids SQL injection).(null safety)

3. Using Sequelize (ORM Example)

ORMs make working with SQL databases easier by allowing you to use **JavaScript syntax** instead of writing raw SQL.

Step 1: Install Sequelize and MySQL2

```
npm install sequelize mysql2
```

Step 2: Setup Connection

```
import { Sequelize, DataTypes } from "sequelize";

const sequelize = new Sequelize("school_db", "root", "password123", {
  host: "localhost",
  dialect: "mysql",
});

try {
  await sequelize.authenticate();
  console.log("Connected to MySQL!");
} catch (error) {
  console.error("Connection failed:", error);
}
```

Step 3: Define a Model (similar to Mongoose Schema)

```
const Student = sequelize.define("Student", {
  name: {
    type: DataTypes.STRING,
    allowNull: false,
  },
  age: {
    type: DataTypes.INTEGER,
    allowNull: false,
  },
});
```

Step 4: Sync and Perform CRUD

```
await sequelize.sync(); // creates table if not exists

// Create
await Student.create({ name: "Bob", age: 25 });

// Read
const students = await Student.findAll();
console.log(students);

// Update
await Student.update({ age: 26 }, { where: { name: "Bob" } });

// Delete
await Student.destroy({ where: { name: "Bob" } });
```

Explanation:

- `sequelize.sync()` creates tables automatically.
- Models represent tables; each row becomes an object's instance.

- CRUD operations are done using simple methods instead of SQL.

4. Best Practices When Working with SQL in Node.js

Practice	Why it matters
Use environment variables (`.env`) for DB credentials	Keeps passwords safe
Use connection pooling	Reuses connections to improve performance
Handle errors properly	Prevents crashes on query failure
Use parameterized queries	Protects against SQL injection
Choose ORM for big projects	Cleaner and more maintainable
Use raw queries for simple/fast operations	More control and less overhead

5. SQL vs. NoSQL (Quick Comparison for Students)

Feature	SQL (MySQL/PostgreSQL)	NoSQL (MongoDB)
Structure	Tables (rows & columns)	Collections (documents)
Schema	Fixed, must define upfront	Flexible, schema-less
Queries	SQL language	JSON-like queries
Relations	Easy to define (JOINs)	Done via references or embedding
Use case	Structured data (banking, users)	Flexible data (blogs, apps)

In Short

To work with SQL databases in Node.js:

1. Install a database driver (like `mysql2`) or ORM (like `Sequelize`).
2. Connect using credentials (from `.env`).
3. Run SQL queries or use model methods for CRUD.
4. Always secure, validate, and organize your database interactions.